A. Output Bubble Sort

```
PS C:\code\Tugas alpro 6 sm2> & "C:\Users\rasyila fuzia\AppData\Local\Programs\Python\Python313\python.exe" "c:/code/Tugas alpro 6 sm2/bubble sort"

Iterasi ke-1:
  Bandingkan 5 dan 2 -> tukar
  Data sekarang: [2, 5, 9, 1, 5, 6]
  Bandingkan 5 dan 9 -> tidak ditukar
  Data sekarang: [2, 5, 9, 1, 5, 6]
  Bandingkan 9 dan 1 -> tukar
  Data sekarang: [2, 5, 1, 9, 5, 6]
  Bandingkan 9 dan 5 -> tukar
  Data sekarang: [2, 5, 1, 5, 9, 6]
  Bandingkan 9 dan 6 -> tukar
  Data sekarang: [2, 5, 1, 5, 6, 9]

Iterasi ke-2:
  Bandingkan 2 dan 5 -> tidak ditukar
  Data sekarang: [2, 5, 1, 5, 6, 9]
  Bandingkan 5 dan 1 -> tukar
  Data sekarang: [2, 1, 5, 5, 6, 9]
  Bandingkan 5 dan 5 -> tidak ditukar
  Data sekarang: [2, 1, 5, 5, 6, 9]
  Bandingkan 5 dan 6 -> tidak ditukar
  Data sekarang: [2, 1, 5, 5, 6, 9]

Iterasi ke-3:
  Bandingkan 2 dan 1 -> tukar
  Data sekarang: [1, 2, 5, 5, 6, 9]
  Bandingkan 2 dan 5 -> tidak ditukar
  Data sekarang: [1, 2, 5, 5, 6, 9]
  Bandingkan 5 dan 5 -> tidak ditukar
  Data sekarang: [1, 2, 5, 5, 6, 9]

Iterasi ke-4:
  Bandingkan 1 dan 2 -> tidak ditukar
  Data sekarang: [1, 2, 5, 5, 6, 9]
  Bandingkan 2 dan 5 -> tidak ditukar
  Data sekarang: [1, 2, 5, 5, 6, 9]

Iterasi ke-5:
  Bandingkan 1 dan 2 -> tidak ditukar
  Data sekarang: [1, 2, 5, 5, 6, 9]

Iterasi ke-6:

Hasil akhir: [1, 2, 5, 5, 6, 9]
Jumlah perbandingan: 15
PS C:\code\Tugas alpro 6 sm2>
```

B. Output Selection Sort

```
PS C:\code\Tugas alpro 6 sm2> & "C:\Users\rasyila fuzia\AppData\Local\Programs\Python\Python313\python.exe" "c:/code/Tugas alpro 6 sm2/selection sort"

Iterasi ke-1:
Mulai dari index 0, asumsi minimum = 5
  Bandingkan 2 dengan 5 -> minimum baru ditemukan: 2
  Bandingkan 9 dengan 2 -> tidak lebih kecil
  Bandingkan 1 dengan 2 -> minimum baru ditemukan: 1
  Bandingkan 5 dengan 1 -> tidak lebih kecil
  Bandingkan 6 dengan 1 -> tidak lebih kecil
Tukar 5 dengan 1
Data sekarang: [1, 2, 9, 5, 5, 6]

Iterasi ke-2:
Mulai dari index 1, asumsi minimum = 2
  Bandingkan 9 dengan 2 -> tidak lebih kecil
  Bandingkan 5 dengan 2 -> tidak lebih kecil
  Bandingkan 5 dengan 2 -> tidak lebih kecil
  Bandingkan 6 dengan 2 -> tidak lebih kecil
Tukar 2 dengan 2
Data sekarang: [1, 2, 9, 5, 5, 6]

Iterasi ke-3:
Mulai dari index 2, asumsi minimum = 9
  Bandingkan 5 dengan 9 -> minimum baru ditemukan: 5
  Bandingkan 5 dengan 5 -> tidak lebih kecil
  Bandingkan 6 dengan 5 -> tidak lebih kecil
Tukar 9 dengan 5
Data sekarang: [1, 2, 5, 9, 5, 6]

Iterasi ke-4:
Mulai dari index 3, asumsi minimum = 9
  Bandingkan 5 dengan 9 -> minimum baru ditemukan: 5
  Bandingkan 6 dengan 5 -> tidak lebih kecil
Tukar 9 dengan 5
Data sekarang: [1, 2, 5, 5, 9, 6]

Iterasi ke-5:
Mulai dari index 4, asumsi minimum = 9
  Bandingkan 6 dengan 9 -> minimum baru ditemukan: 6
Tukar 9 dengan 6
Data sekarang: [1, 2, 5, 5, 6, 9]

Iterasi ke-6:
Mulai dari index 5, asumsi minimum = 9
Tukar 9 dengan 9
Data sekarang: [1, 2, 5, 5, 6, 9]

Hasil: [1, 2, 5, 5, 6, 9]
Jumlah perbandingan: 15
PS C:\code\Tugas alpro 6 sm2>
```

C. Output Insertion Sort

```
PS C:\code\Tugas alpro 6 sm2> & "C:\Users\rasyila fuzia\AppData\Local\Programs\Python\Python313\python.exe" "c:/code/Tugas alpro 6 sm2/insertion sort"

Iterasi ke-1:
Ambil key = 2
  5 > 2 -> geser 5 ke kanan
  Data sekarang: [5, 5, 9, 1, 5, 6]
Masukkan key di posisi 0
Data sekarang: [2, 5, 9, 1, 5, 6]

Iterasi ke-2:
Ambil key = 9
  5 <= 9 -> berhenti geser
Masukkan key di posisi 2
Data sekarang: [2, 5, 9, 1, 5, 6]

Iterasi ke-3:
Ambil key = 1
  9 > 1 -> geser 9 ke kanan
  Data sekarang: [2, 5, 9, 9, 5, 6]
  5 > 1 -> geser 5 ke kanan
  Data sekarang: [2, 5, 5, 9, 5, 6]
  2 > 1 -> geser 2 ke kanan
  Data sekarang: [2, 2, 5, 9, 5, 6]
Masukkan key di posisi 0
Data sekarang: [1, 2, 5, 9, 5, 6]

Iterasi ke-4:
Ambil key = 5
  9 > 5 -> geser 9 ke kanan
  Data sekarang: [1, 2, 5, 9, 9, 6]
  5 <= 5 -> berhenti geser
Masukkan key di posisi 3
Data sekarang: [1, 2, 5, 5, 9, 6]

Iterasi ke-5:
Ambil key = 6
  9 > 6 -> geser 9 ke kanan
  Data sekarang: [1, 2, 5, 5, 9, 9]
  5 <= 6 -> berhenti geser
Masukkan key di posisi 4
Data sekarang: [1, 2, 5, 5, 6, 9]

Hasil: [1, 2, 5, 5, 6, 9]
Jumlah perbandingan: 9
PS C:\code\Tugas alpro 6 sm2>
```

D. Perbandingan Tiga Sorting

Jumlah Data	Bubble	Selection	Insertion
6	15	15	9
10	45	45	30
20	190	190	81

Analisis:

1. Paling sedikit perbandingan

   Insertion Sort, karena:

   - Bisa jauh lebih sedikit jika data sudah hamper terurut
   - Bahkan bisa mendekati n-1 saja

2. Paling stabil pada data hamper terurut

   Insertion Sort, karena:

   - Hanya menggeser sedikit elemen
   - Tidak perlu banyak perbandingan
   - Tidak melakukan swap besar seperti bubble

E. Challenge

1. Output Insertion Sort pada data terurut

```
PS C:\code\Tugas alpro 6 sm2> & "C:\Users\rasyila fuzia\AppData\Local\Programs\Python\Python313\python.exe" "c:/code/Tugas alpro 6 sm2/insertion sort pada data ham
pir terurut"
===== DATA HAMPIR TERURUT =====
Data awal: [1, 2, 3, 5, 4, 6]

Iterasi ke-1:
Ambil key = 2
  1 <= 2 -> berhenti
Masukkan key di posisi 1
Data sekarang: [1, 2, 3, 5, 4, 6]

Iterasi ke-2:
Ambil key = 3
  2 <= 3 -> berhenti
Masukkan key di posisi 2
Data sekarang: [1, 2, 3, 5, 4, 6]

Iterasi ke-3:
Ambil key = 5
  3 <= 5 -> berhenti
Masukkan key di posisi 3
Data sekarang: [1, 2, 3, 5, 4, 6]

Iterasi ke-4:
Ambil key = 4
  5 > 4 -> geser 5 ke kanan
  Data sekarang: [1, 2, 3, 5, 5, 6]
  3 <= 4 -> berhenti
Masukkan key di posisi 3
Data sekarang: [1, 2, 3, 4, 5, 6]

Iterasi ke-5:
Ambil key = 6
  5 <= 6 -> berhenti
Masukkan key di posisi 5
Data sekarang: [1, 2, 3, 4, 5, 6]

Hasil: [1, 2, 3, 4, 5, 6]
Jumlah perbandingan: 6
PS C:\code\Tugas alpro 6 sm2>
```

2. Output Insertion Sort pada data acak

```
PS C:\code\Tugas alpro 6 sm2> & "C:\Users\rasyila fuzia\AppData\Local\Programs\Python\Python313\python.exe" "c:/code/Tugas alpro 6 sm2/insertion sort pada data aca
k"
===== DATA ACAK =====
Data awal: [5, 2, 9, 1, 5, 6]

Iterasi ke-1:
Ambil key = 2
  5 > 2 -> geser 5 ke kanan
  Data sekarang: [5, 5, 9, 1, 5, 6]
Masukkan key di posisi 0
Data sekarang: [2, 5, 9, 1, 5, 6]

Iterasi ke-2:
Ambil key = 9
  5 <= 9 -> berhenti
Masukkan key di posisi 2
Data sekarang: [2, 5, 9, 1, 5, 6]

Iterasi ke-3:
Ambil key = 1
  9 > 1 -> geser 9 ke kanan
  Data sekarang: [2, 5, 9, 9, 5, 6]
  5 > 1 -> geser 5 ke kanan
  Data sekarang: [2, 5, 5, 9, 5, 6]
  2 > 1 -> geser 2 ke kanan
  Data sekarang: [2, 2, 5, 9, 5, 6]
Masukkan key di posisi 0
Data sekarang: [1, 2, 5, 9, 5, 6]

Iterasi ke-4:
Ambil key = 5
  9 > 5 -> geser 9 ke kanan
  Data sekarang: [1, 2, 5, 9, 9, 6]
  5 <= 5 -> berhenti
Masukkan key di posisi 3
Data sekarang: [1, 2, 5, 5, 9, 6]

Iterasi ke-5:
Ambil key = 6
  9 > 6 -> geser 9 ke kanan
  Data sekarang: [1, 2, 5, 5, 9, 9]
  5 <= 6 -> berhenti
Masukkan key di posisi 4
Data sekarang: [1, 2, 5, 5, 6, 9]

Hasil: [1, 2, 5, 5, 6, 9]
Jumlah perbandingan: 9
PS C:\code\Tugas alpro 6 sm2>
```

3. **Perbedaan Insertion Sort pada data hamper terurut dan dengan data acak**

   Perbedaan utama antara data hampir terurut dan data acak pada Insertion Sort terletak pada jumlah pergeseran (shift) dan perbandingan yang dilakukan.

   a. Pada data hamper terurut:
      - Elemen sudah hampir di posisi benar
      - Hanya sedikit pergeseran
      - Perbandingan jadi lebih sedikit
   b. Pada data acak:
      - Banyak elemen harus dipindah
      - Terjadi banyak pergeseran
      - Perbandingan jadi lebih banyak

F. Kesimpulan

Ketiga algoritma sorting yaitu Bubble Sort, Selection Sort, dan Insertion Sort memiliki cara kerja yang berbeda dalam mengurutkan data. Bubble Sort bekerja dengan membandingkan elemen bersebelahan dan menukarnya berulang kali hingga data terurut, tetapi cenderung kurang efisien karena banyak melakukan pertukaran. Selection Sort memilih elemen terkecil dari bagian yang belum terurut lalu menukarnya ke posisi depan, dengan jumlah perbandingan yang tetap meskipun data sudah hampir terurut. Insertion Sort menyisipkan elemen ke posisi yang tepat dengan menggeser elemen sebelumnya, sehingga lebih efisien terutama jika data sudah hampir terurut. Dari segi jumlah perbandingan, Bubble dan Selection memiliki jumlah yang sama pada kondisi umum, sedangkan Insertion bisa lebih sedikit tergantung kondisi data. Selain itu, Insertion Sort lebih adaptif dan stabil dibanding dua algoritma lainnya. Oleh karena itu, Insertion Sort sering menjadi pilihan yang lebih baik untuk data berukuran kecil atau hampir terurut.